

Ansible 발표 필수 개념 — 학습 노트

이 문서의 목적: 본인이 Ansible 발표 중 자연스럽게 답변하고, 슬라이드 본문 외 추가 질문이 들어와도 막히지 않도록 핵심 개념 + 우리 프로젝트 적용 사례를 묶어둠.

사용 방법:

1. 발표 전 한 번 정독 (개념 흐름 파악)
2. 슬라이드 작성 중 "이 부분 말로 어떻게 설명하지?" 막히면 §6 (예상 질문) 참조
3. 발표 직전 §7 (자연스럽게 쓸 표현) 만 다시 훑어보기

이 문서는 발표 자료 X. 본인 머릿속 정리용.

1. Ansible 이 뭐예요? — 1줄 정의

한 문장: "사람이 SSH 로 들어가서 명령어 치는 걸 자동화하는 도구."

보강 (필요 시 추가): "Configuration Management 도구 — OS 위에서 패키지 설치 / 파일 배포 / 서비스 시작/정지 / 사용자 관리 같은 일들을 YAML 코드로 선언."

왜 "자동화" 가 핵심인가

사람이 SSH 들어가서 명령어 치는 방식의 문제:

- 호스트 100대면 100번 반복 (시간 낭비)
- 사람마다 약간씩 다르게 침 (snowflake 서버 — 운영 지옥의 시작)
- 기록이 안 남음 (왜 이렇게 됐는지 사후 추적 불가)
- 검증 안 됨 (했다고 생각했는데 안 됐을 수도)

Ansible 이 해결:

- YAML 한 번 쓰면 100대 동일 적용
- 코드 = 기록 (git history 가 변경 이력)
- 멍등성 (다시 돌려도 안전 — §3-4 참조)

2. Terraform vs Ansible — 영역이 다름

발표 중 가장 자주 나오는 혼동. 두 도구는 경쟁 X, 보완 O.

	Terraform	Ansible
다루는 것	인프라 자체 (EC2, RDS, ALB, VPC)	OS 위 설정 (패키지, 파일, 서비스, 사용자)
시점	"EC2 가 있어야 한다"	"EC2 안에 Java 17 이 깔려 있어야 한다"
언어	HCL (HashiCorp 전용)	YAML
Push/Pull	Cloud API 호출 (Push)	SSH 로 들어가서 실행 (Push)

	Terraform	Ansible
State	tfstate 파일 보유	무상태 (매번 호스트 직접 확인)
우리 프로젝트	AWS 자원 생성	온프레임 VM 셋업 + 앱 배포

발표 답변 패턴

"Terraform 으로 EC2 라는 빈 박스를 만들고, Ansible 로 그 박스 안에 Java + Spring Boot 를 채워넣습니다. 한 도구로 두 영역을 다 하려면 어느 한 쪽이 어색해져요. 우리는 각 도구를 가장 잘하는 영역에 씁니다."

3. 핵심 개념 5가지 — 발표에서 반드시 알고 있어야 할 것

3-1. Inventory — "어디에 적용할 것인가"

무엇: 호스트 목록 + 그룹 분류 파일.

우리 프로젝트: [Ansible/inventories/on-premise/hosts.yml](#)

- 그룹 3개: `webwas` / `db` / `haproxy`
- 각 그룹에 호스트 (IP, SSH 사용자 정보 등)

왜 그룹 분리: 같은 명령을 "`webwas` 그룹 전체에" 던질 수 있게. 호스트 추가 시 그룹에만 넣으면 끝.

환경별 inventory 분리:

- `inventories/on-premise-phase1/` — Phase 1 머신용
- `inventories/on-premise/` — Phase 3 머신용

→ 같은 playbook 을 다른 inventory 로 던지면 다른 환경에 적용. **playbook 코드는 변경 0.**

3-2. Playbook — "무엇을, 어디에 적용할 것인가"

무엇: YAML 파일. "`이 호스트 그룹에 이 role 들을 적용하라`" 는 명세.

우리 프로젝트:

- [Ansible/playbooks/site.yml](#) — 전체 6 roles 분산 적용 (Phase 3 풀 셋업)
- [Ansible/playbooks/webwas.yml](#) — `webwas` 호스트만 3 roles (Phase 1 앱 갱신)
- [Ansible/playbooks/db_failback.yml](#) — Failback 시 RDS dump → onprem restore 전용

핵심 통찰: Playbook 은 "`무엇을 할지의 contract`" — 같은 contract 를 다른 inventory 에 던지거나, 같은 inventory 에 다른 contract 를 던질 수 있음. 조합 가능.

3-3. Role — "재사용 가능한 단위"

무엇: 표준 폴더 구조를 가진 자동화 모듈. 한 번 잘 만들면 여러 playbook 에서 재사용.

우리 프로젝트 6 roles ([Ansible/roles/](#)):

Role	책임
common	모든 호스트 공통 (방화벽, 사용자, timezone 등)
haproxy	HAProxy 설치 + config
tailscale	Tailscale 설치 + subnet router 설정 (HAProxy 만)
mysql	MySQL 8.0 설치 + 사용자 생성
springboot	Java 설치 + jar 배포 + systemd 서비스 등록
cloudwatch-agent	CloudWatch Agent 설치 + 메트릭 수집

Role 폴더 구조 (각 role 마다):

```
roles/<role-name>/
├── tasks/main.yml      # 실행할 일들 (핵심)
├── handlers/main.yml   # 변경 시만 실행 (예: config 바뀌면 service restart)
├── templates/*.j2      # Jinja2 템플릿 (변수 치환되는 파일)
├── defaults/main.yml   # 변수 기본값 (가장 낮은 우선순위)
├── vars/main.yml       # 고정 변수 (높은 우선순위)
└── files/              # 정적 파일 (그대로 복사할 것들)
```

왜 이 구조: 책임 분리 + 재사용. "이 role 안에서 일어나는 모든 것" 이 한 폴더에 모임 → 다른 프로젝트에 그대로 복사 해도 동작.

3-4. 멍등성 (Idempotency) — DR 의 생존 조건

한 줄 정의: 같은 playbook 을 100번 실행해도 100번 모두 같은 결과.

구현 원리:

- Ansible 이 "원하는 상태" (예: `package: present`) 를 받음
- "현재 상태" 확인 (예: 패키지 깔려있나?)
- diff 만 적용 (이미 깔려있으면 skip, 없으면 install)

왜 DR 에서 결정적인가 — 3가지 이유:

1. **재시도 가능:** 네트워크 끊김 / 일부 실패 → 처음부터 다시 돌려도 안전. "한 번에 끝까지 못 돌려도 되는 자동화".
2. **부분 실패 복구:** 3대 호스트 중 1대만 실패 → 1대만 재실행해도 다른 2대 영향 없음.
3. **검증 가능:** 실행 후 "변경 0건" 보고 = "이미 원하는 상태" 확인 = 정상 상태 보장.

발표 표현 (외워두면 좋음):

"Ansible 의 멍등성은 DR 의 안전망입니다. 운영자가 두려움 없이 재실행할 수 있어야 진짜 자동화입니다 — 한 번만 실행 가능한 스크립트는 자동화가 아니라 일회성 도구입니다."

3-5. 변수 우선순위 — 6단계 (낮 → 높)

자동화의 함정이 가장 자주 숨는 곳. **narrative 3** (`ansible_user` 누락) 이 정확히 이 영역.

순서	위치	우선순위
1	기본값 (Ansible 내장 — 예: <code>ansible_user</code> = 현재 OS 사용자)	낮음
2	<code>group_vars/all.yml</code>	
3	<code>group_vars/<group>.yml</code> (예: <code>webwas.yml</code>)	
4	<code>host_vars/<host>.yml</code>	
5	inventory 파일의 host 레벨 정의	
6	CLI <code>-e "var=value"</code>	높음

원칙: 위로 갈수록 "구체적이고 우선". 같은 변수가 여러 곳에 있으면 더 구체적인 쪽이 이김.

우리 프로젝트 사례:

- `ansible_user`: `ansible` → `group_vars/all.yml` (모든 호스트 공통)
- `cloudwatch_enable_procstat`: `true` → `group_vars/webwas.yml` (webwas 만)
- `tailscale_advertise_routes`: `"192.168.20.0/24"` → `group_vars/haproxy.yml` (haproxy 만)

→ 호스트별 다른 동작이 필요하면 `group_vars` / `host_vars` 단계에서 분기.

4. 우리 프로젝트의 아키텍처 elegance — "같은 코드, 다른 시나리오"

4-1. 본인의 원래 디자인 의도

"Phase 3 에서의 1-Click 자동 복구를 최대한 구현하기 위해 [Ansible 을] 최소한으로만 사용하려고 했습니다."

처음에는 Phase 3 (Failback) 만을 위해 Ansible 코드를 짰음. "AWS 에서 온프레임으로 돌아갈 때 한 번만 동작하면 된다" 는 의도.

4-2. 디자인 변경의 trigger

WEB-WAS 의 변경:

- 원래 기획: Nginx + Tomcat (둘 다 별도 셋업 필요, 설정 파일 많음)
- 변경: Spring Boot 단일 jar (jar 파일 하나만 갈아끼우면 끝)

그 결과:

- Spring Boot jar 갱신은 "단순 파일 복사 + 서비스 재시작" 수준
- → Phase 3 Ansible role 의 `springboot` 부분이 그대로 Phase 1 앱 갱신에도 사용 가능

4-3. 결정 — Phase 1 도 Jenkins + Ansible 로

기존 Phase 1 = 수동 배포 → 변경: Jenkins 가 `webwas.yml` 실행.

왜 가능한가 — 두 가지 조건이 동시에 충족:

1. **Spring Boot 단일 jar** 가 "앱 갱신" 을 "단순 작업" 으로 만들
2. **Ansible 의 멍등성** 이 "같은 role 을 부분 재실행" 해도 안전하게 만들

→ Phase 1 / Phase 3 가 **같은 role 코드를 재사용**. 코드 중복 0%.

4-4. 발표 표현 (정수 메시지)

"Spring Boot 단일 jar 선택이 IaC 단순화로 이어졌습니다. 같은 Ansible role 을 Phase 3 에서는 풀 셋업으로, Phase 1 에서는 앱 업데이트로 재사용합니다. 이게 가능한 이유는 **역등성** — Ansible 의 본질적 속성이 우리 아키텍처를 단순하게 만들어준 거죠."

왜 이 메시지가 강한가:

- 단순한 "우리 시스템 좋아요" 가 아니라
- "기술 선택이 어떻게 다른 기술 선택을 단순화했는가" 의 인과 관계 설명
- 청중이 "이 사람은 도구를 그냥 쓰는 게 아니라 도구 간 상호작용을 이해한다" 인식

5. 추가 개념 (시간 여유 있을 때 알아두면 좋은 것)

5-1. Handler — "변경 시만 실행"

문제 상황: config 파일을 매번 다시 쓰면 매번 service restart 가 일어남. 하지만 실제 config 가 안 바뀌었으면 restart 불필요.

해결: Task 가 "실제로 변경" 발생 시 → notify 를 통해 handler 호출 → handler 가 service restart.

```
# tasks/main.yml
- name: Deploy HAProxy config
  template:
    src: haproxy.cfg.j2
    dest: /etc/haproxy/haproxy.cfg
  notify: Restart HAProxy # ← 변경 발생 시만

# handlers/main.yml
- name: Restart HAProxy
  systemd:
    name: haproxy
    state: restarted
```

→ **변경 없으면 restart 도 없음** = 운영 안정성 + 역등성 강화.

5-2. Jinja2 Template — "변수 치환되는 파일"

.j2 확장자 = 텍스트 파일 + 변수 치환 + 조건문/반복문.

```
# templates/springboot.service.j2
[Service]
ExecStart=/usr/bin/java {{ spring_boot_jvm_opts }} -jar {{ spring_boot_jar_path }}
Environment=APP_ENV={{ app_env }}
{% if cloudwatch_enabled %}
```

```
Environment=CW_ENABLED=true
{% endif %}
```

→ Ansible 이 변수 채워서 최종 파일 생성.

우리 프로젝트 사례: `roles/cloudwatch-agent/templates/amazon-cloudwatch-agent.json.j2` — 호스트 별로 다른 메트릭 설정 생성.

5-3. Ansible Vault — "비밀번호 암호화"

문제: 비밀번호를 yaml 에 평문으로 쓰면 git 에 그대로 올라감.

해결: `ansible-vault encrypt` 로 암호화된 yaml 생성. playbook 실행 시 vault password 입력 → 자동 복호화.

우리 프로젝트: `vault_db_app_password` 같은 변수가 vault 에 저장됨. 코드에는 변수 참조만 보이고 실제 값은 git 에 평문으로 안 올라감.

6. 예상 질문 6개 — Q&A 대비

Q1. "Ansible 과 Terraform 의 차이가 뭔가요?"

A: "Terraform 은 인프라 자체 (EC2, RDS) 를, Ansible 은 OS 위 설정 (패키지, 파일, 서비스) 를 다룹니다. 영역이 다르니 보완 관계예요. 우리는 Terraform 으로 EC2 빈 박스 만들고, Ansible 로 그 안에 Java/Spring Boot 채워넣습니다."

Q2. "역등성이 뭐고 왜 중요한가요?"

A: "같은 명령을 N번 실행해도 같은 결과가 나오는 성질입니다. DR 에서 결정적인 이유는 세 가지 — 재시도 가능, 부분 실패 복구, 검증 가능. 운영자가 두려움 없이 재실행할 수 있어야 진짜 자동화죠."

Q3. "role 은 어떻게 구성되어 있나요?"

A: "표준 폴더 구조 — `tasks` (실행할 일), `handlers` (변경 시만), `templates` (Jinja2), `defaults` (기본값), `vars` (고정값), `files` (정적 파일). 책임이 한 폴더에 모여있어서 다른 프로젝트로 그대로 복사해도 동작합니다."

Q4. "inventory 와 playbook 의 관계는?"

A: "Inventory 는 어디에, playbook 은 무엇을. 분리되어 있어서 같은 playbook 을 다른 inventory 에 던지면 다른 환경 (Phase 1 / Phase 3) 에 적용됩니다. 코드는 변경 0."

Q5. "비밀번호는 어디에 두나요?"

A: "Ansible Vault — 암호화된 yaml. 우리 프로젝트도 DB 비밀번호 같은 secrets 가 vault 에 들어가있어서 git 에 평문으로 안 올라갑니다. playbook 실행 시 vault 패스워드만 입력하면 자동 복호화."

Q6. "왜 Ansible 인가요? Bash script 로 충분하지 않나요?"

A: "Bash 도 가능하지만 역등성을 직접 구현해야 하고, 호스트 여러 대 관리 시 ssh 루프를 직접 돌려야 합니다. 변수 관리, 템플릿 처리, 에러 핸들링 모두 직접 구현해야 하죠. Ansible 이 이것 다 표준화했어요. YAML 한 번 쓰면 100대 동일 적용 — 그래서 인프라 자동화 표준으로 자리잡았습니다."

7. 발표 중 자연스럽게 쓸 표현 모음

외울 필요는 없고, 흐름상 자연스럽게 나오면 좋은 어휘들:

- "역동성을 보장하는 자동화"
- "같은 코드, 다른 시나리오"
- "Role 분리로 책임 격리"
- "호스트 그룹별 역할 분산"
- "변수 계층으로 환경별 차이 흡수"
- "선언적 도구의 안전망"
- "Configuration drift 방어" (snowflake 서버 예방)
- "Push 모델의 단순함" (agent 설치 불필요 — SSH 만 있으면 됨)

8. 한 가지 주의 — 발표 중 절대 하지 말 것

❌ **너무 깊이 들어가지 말 것.** 발표 청중이 비엔지니어일 수 있고, Ansible 자체가 발표 주제가 아님 (DR 시스템이 주제).

✅ **묻는 만큼만 답하고,** 더 깊은 질문 들어오면 "이 부분은 발표 후에 별도로 설명드릴 수 있습니다" 로 매듭. "내가 다 안다" 보다 "필요한 만큼 안다" 가 신뢰 줌.

특히 다음 영역은 발표 후 별도 자리:

- Ansible Plugin / Module 개발
- Dynamic inventory
- Custom filter / lookup
- Ansible Tower / AWX

→ 우리 프로젝트는 위 영역 안 씬. 굳이 언급할 필요 X.

마지막 한 마디: Ansible 은 "YAML 잘 짜는 사람" → "운영을 자동화하는 사람" 으로 가는 도구. 본인은 이미 후자에 발 디딘 상태입니다. 자신감 가지셔도 됩니다.